

Arwa Mahwash 2023-CS-16

Compiler Construction Lab

1 Introduction

This report documents symbol-table implementation. The module supports a fixed-size hash table, nested scopes, shadowing, duplicate checks, undeclared-name detection, and tabular scope dumps. It is written in Python and can be run independently through the included driver.

2 Assignment Goals

The lab requires five major outcomes:

- Build a hash table for one scope with insert, lookup, delete, and print.
- Add scope stacking with begin-scope and end-scope behavior.
- Detect duplicate declarations in the same scope and undeclared uses.
- Pretty-print each scope in a table that is easy to inspect.
- Provide a short test report with examples and observed output.

3 Implementation Summary

3.1 Table Structure

The symbol table uses 211 buckets, which matches the lab specification. Each bucket stores a linked chain of entries, so collisions are handled by separate chaining instead of replacement. This keeps the implementation simple while still supporting multiple names that hash to the same slot.

3.2 Insertion and Lookup

Insertion checks only the current scope before adding a new name. If the name already exists in the current scope, the implementation writes an error message to standard error and rejects the declaration. Lookup first checks the current scope and then climbs through parent scopes until a match is found or the global scope is reached.

3.3 Scope Handling

Each scope stores a pointer to its parent and a numeric scope level. Opening a scope creates a new table whose parent is the previous current scope. Closing a scope prints the contents of the current scope before returning the parent scope. This behavior is important because the lab requires the popped scope to be dumped before it disappears.

3.4 Pretty Printing

The table printer produces a compact aligned view with the columns ID, Name, Kind, Type, Scope, and Line. The output is sorted by line number and insertion order so the dump is easy to compare with the driver output and with the expected lab format.

3.5 Error Reporting

The implementation reports two important error conditions:

- Duplicate declaration in the same scope.
- Use of an undeclared variable.

These messages are written in the same style as the sample lab output, which makes them suitable for grading and manual inspection.

4 Files in This Folder

The alternate submission is split into a small, clear folder structure.

- `src/scope_table.py` contains the hash table, scope stack, lookup logic, deletion, and printing.
- `demo_driver.py` runs two scenarios that exercise the implementation.
- `test/` contains five input fixtures.
- `output/` contains the corresponding captured outputs.

5 Test Cases

Five fixtures were created in the `test/` folder and their outputs were saved in the `output/` folder. The table below describes what each case checks.

Table 1: Test cases for the alternate submission

Case	Fixture	Expected Result	Observed Result
1	<code>alpha_basic_hash.txt</code>	10 inserts, 5 lookups, and 3 deletes on one scope.	The final table contains 7 names after deletion and the lookup results include both hits and misses.
2	<code>beta_shadowing.txt</code>	Inner <code>x</code> hides outer <code>x</code> in nested scopes.	Lookup resolves to scope 0 first and scope 2 after shadowing.
3	<code>gamma_errors.txt</code>	Duplicate and undeclared-name errors are reported.	Error lines for duplicate <code>x</code> and undeclared <code>a</code> were printed.
4	<code>delta_scope_dump.txt</code>	Each scope is dumped when it ends.	Scope 2, then scope 1, then scope 0 were printed in exit order.

Case	Fixture	Expected Result	Observed Result
5	epsilon_table_format.txt	Tabular print is aligned and readable.	The output includes a header row and aligned columns.

6 Observed Output

The most important runtime behavior is shown below.

```
[Scope 0] Enter
[Scope 1] Enter
[Scope 2] Enter
[Scope 2] lookup x -> found at scope 0
[Scope 2] lookup y -> found at scope 0
[Scope 2] lookup x -> found at scope 2 after shadowing
[Scope 2] Exit, dump:
ERROR line 11: duplicate declaration 'x'
ERROR line 12: undeclared variable 'a'
[Scope 0] Exit, dump:
```

7 Limitations and Notes

The driver demonstrates the symbol table directly, so parser integration is described in the report rather than hard-coded into the test script.

8 Conclusion

The test fixtures and output files included in this folder provide evidence that the behavior matches the assignment requirements.